Pre-Trained Hyper-Heuristics for Optimisation Tasks

University of Exeter

Made by Sandy Hay

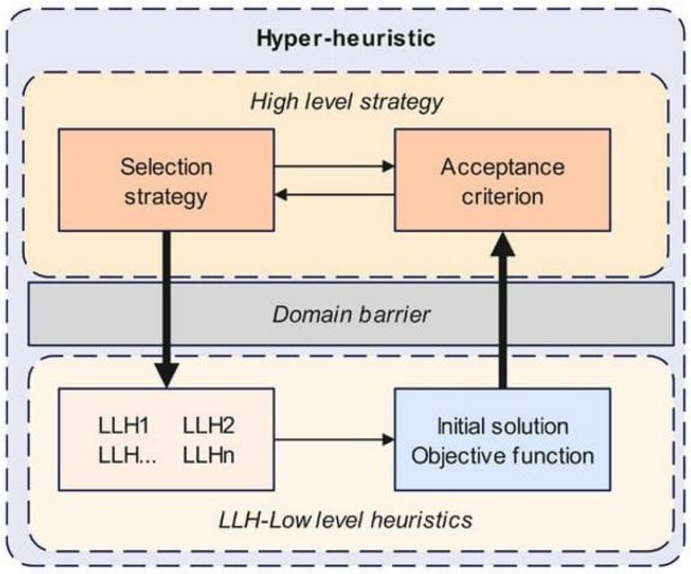
Motivation

- **Transfer Learning Between Problem Instances:** Allows hyper-heuristic exploit previously learning heuristic performance patterns
- **Improving Early-Stage Solution Quality:** Helps hyper-heuristics prioritise heuristics that work well with similar problem domains
- **Stabilise Heuristic Selection Behaviour:** Heuristic Selection is highly stochastic. Learns consistent patterns across multiple training runs

Aims and Objectives

- Create and implement a hyper-heuristic that operates on the problem domains in Hyflex [1], using low-level heuristic selection and acceptance methods.
- Attach a Python-Java Gateway that runs Hyflex hyper-heuristics on the problem domains to visually compare results across multiple runs and instances.
- Investigate whether pre-training a hyper-heuristic on known problem domains can transfer prior heuristic performance knowledge to improve solution quality on unseen instances.

Methodology

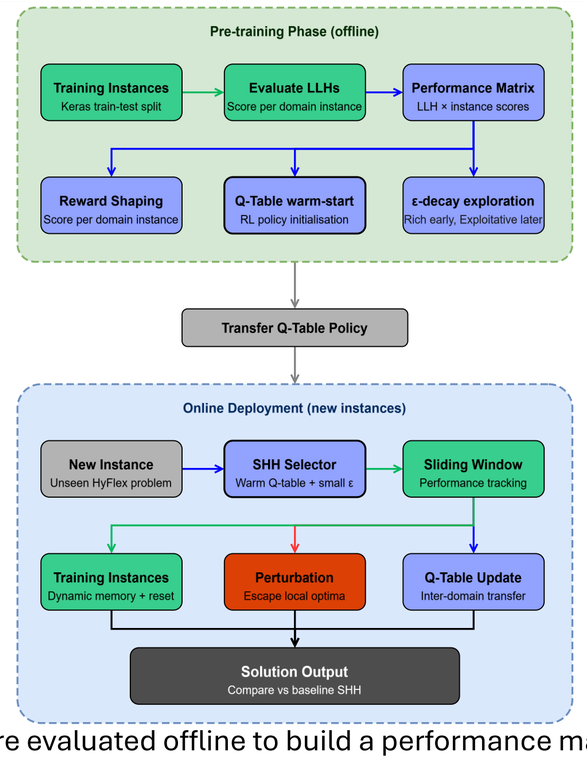


[2] Generic Hyper-Heuristic Structure

1. Built using a Choice Function Selection Hyper-heuristic (SHH).
The heuristic selection score function is:

$$score(h) = a \cdot F_1(h) + \beta \cdot F_2(prev, h) + y \cdot F_3(h)$$

2. The Choice Function HH base class handles core functionality. **The Pre-Trained HH** extends this with offline Keras training and RL with a Q-table. **The Py4J Bridge** connects Python selection to HyFlex Java domains. **All hyper-heuristics run on the same problem domains.**

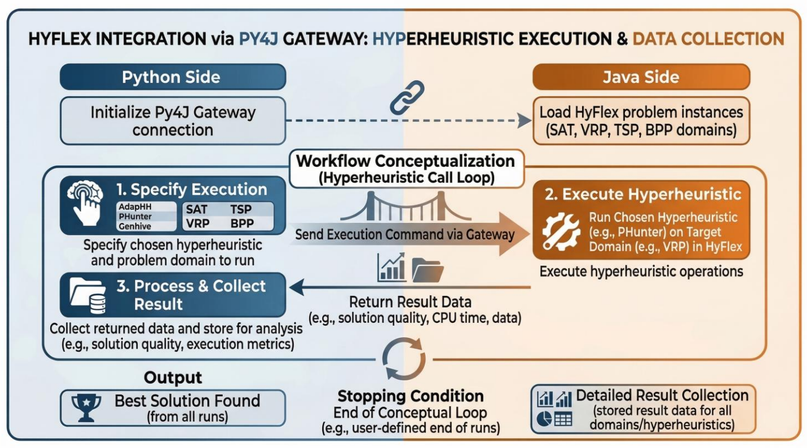


3. LLHs are evaluated offline to build a performance matrix, warm-starting the Q-table and shaping rewards. The policy transfers to online deployment, where the SHH selector adapts via sliding window tracking, perturbation for escaping local optima, and continuous Q-table updates.

Implementation

Py4J connects the Python hyper-heuristic to the Java HyFlex framework at runtime. Python drives the learning loop — selecting a heuristic via the Choice Function, sending it to Java for execution, and receiving the updated fitness to adjust the Q-table. All machine learning (ML) logic stays in Python while HyFlex handles problem domain execution.

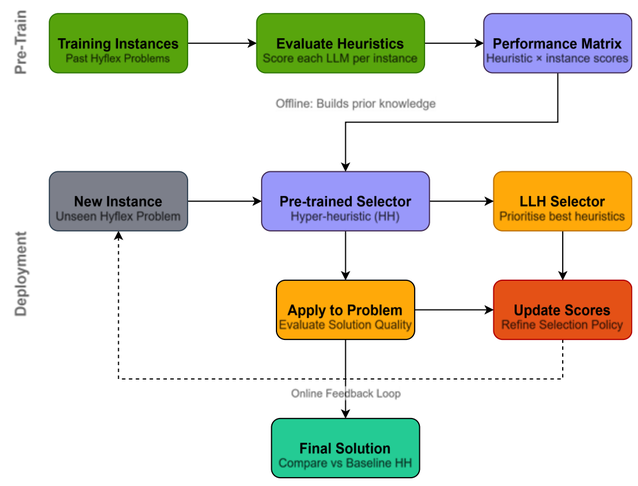
Py4J Java Gateway



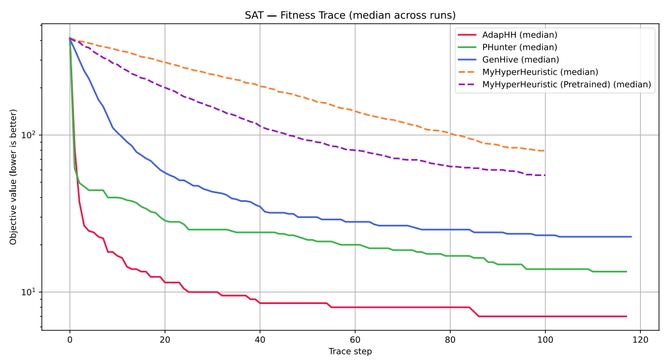
Pre-Training Hyper-heuristic

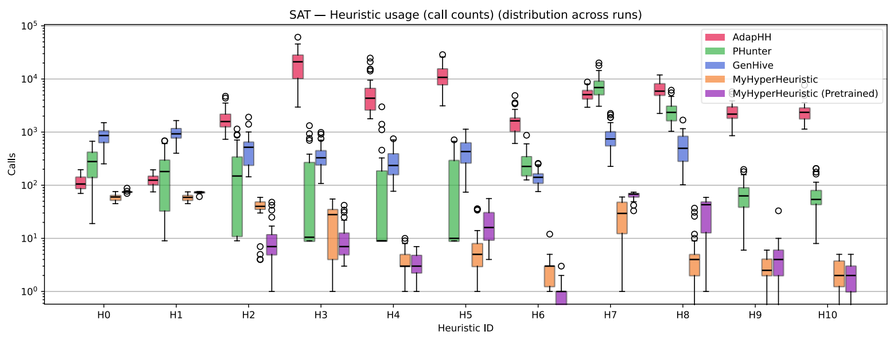
Two-phase pipeline separate offline pre-training from online deployment. Pre-training evaluates low-level heuristics across known problem instances to build a performance matrix.

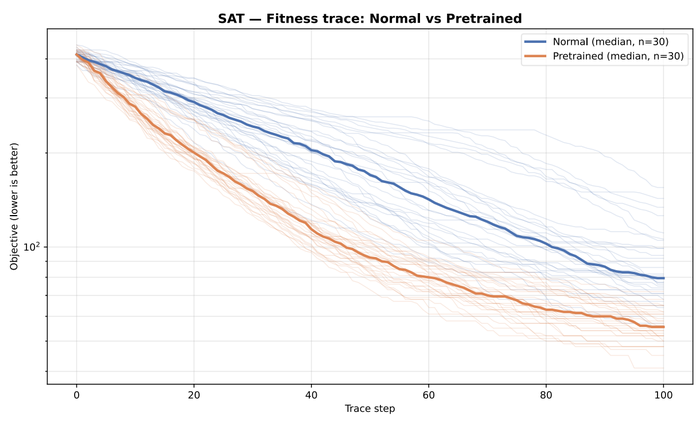
At deployment, pre-training selector warm-starts hyper-heuristic and applies known knowledge to unseen problems.

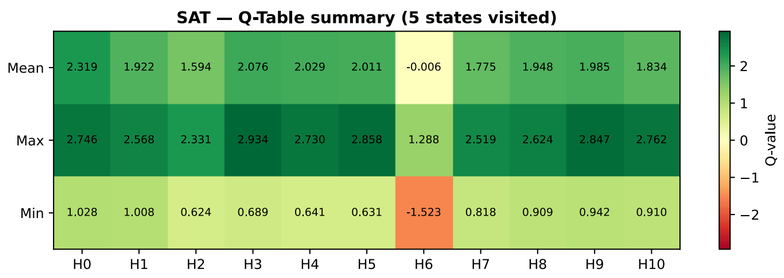


Results and Conclusion









- Pretraining a hyper-heuristic is achievable, can perform better than the standard hyper-heuristic
- It is very hard to avoid stochastic fitnesses due to varying instances and seeds allowing
- Problem domains have different responses to pretraining:
  - SAT generalises well to pretraining, having improved over time and outperforming the normal fitness across each step
  - VRP approximately converges 50% faster with pretraining
  - TSP seems to be unaffected by the pretraining due to the problem being too complex
  - Bin Packing also generalises well to the pretraining

References

[1] - seage. 2025. "GitHub - Seage/Hyflex: Some Hyper-Heurisics from CHeSC 2011 Challenge and the Challenge Results Reproducing Attempts." GitHub. 2025. https://github.com/seage/hyflex.
[2] - Dang, Yinglong, Xiaoguang Gao, and Zidong Wang. "Choice Function-Based Hyper-Heuristics for Causal Discovery under Linear Structural Equation Models." *Biomimetics* 9, no. 6 (2024): 350.